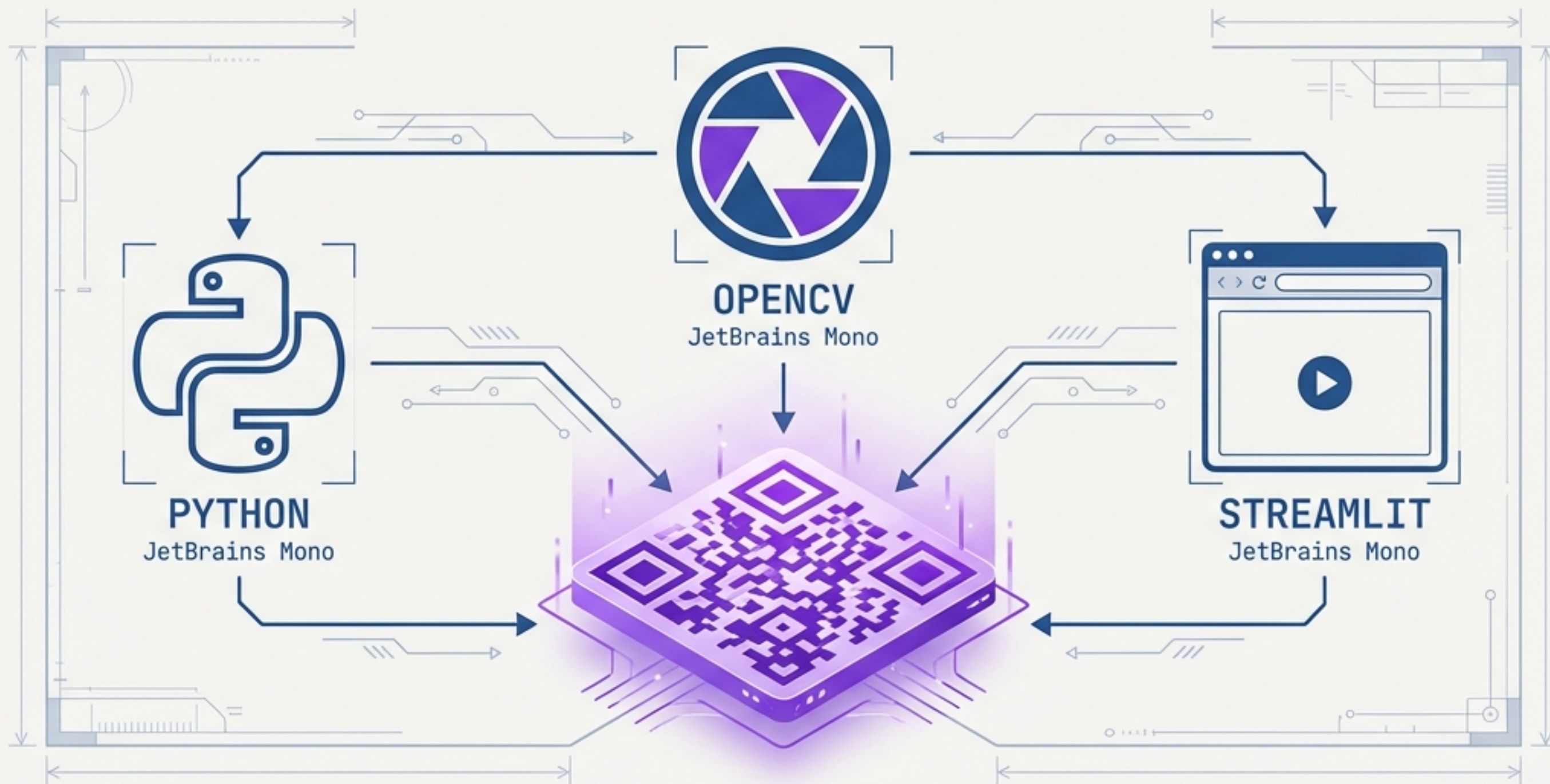


AI-QR Master: 파이썬으로 만드는 컴퓨터 비전 웹앱

코드 한 줄에서 시작해 견고한 제품을 완성하는 엔지니어링 여정



우리의 무기: 강력한 3가지 라이브러리



Streamlit

웹 프론트엔드?
HTML/CSS 없이 파이썬만으로
가장 빠르게 UI를 구축합니다.



OpenCV (headless)

컴퓨터 비전의 표준.
이미지를 숫자로 변환하고
조작하는 강력한 '눈'입니다.



Pyzbar

QR 및 바코드를 0.1초 만에
해독하는 전용 디코딩
엔진입니다.

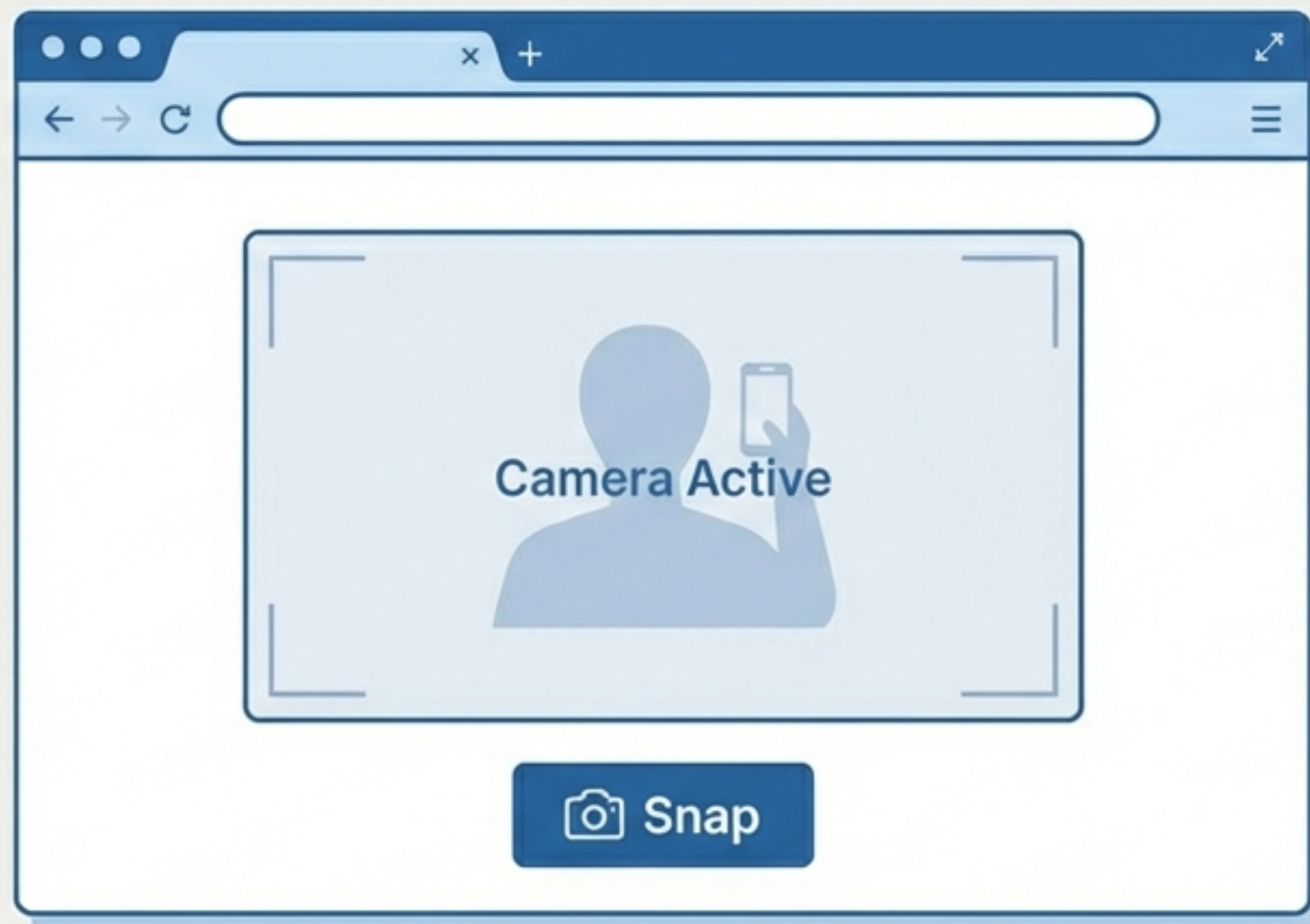
```
pip install streamlit opencv-python-headless pyzbar
```

💡 **Insight:** 'Headless' 버전의 OpenCV는 서버 환경에서 가볍고 빠르게 돌아가기 위해 GUI 기능을 뺀 버전입니다.

Step 1: 헬로 월드 & 카메라 연동

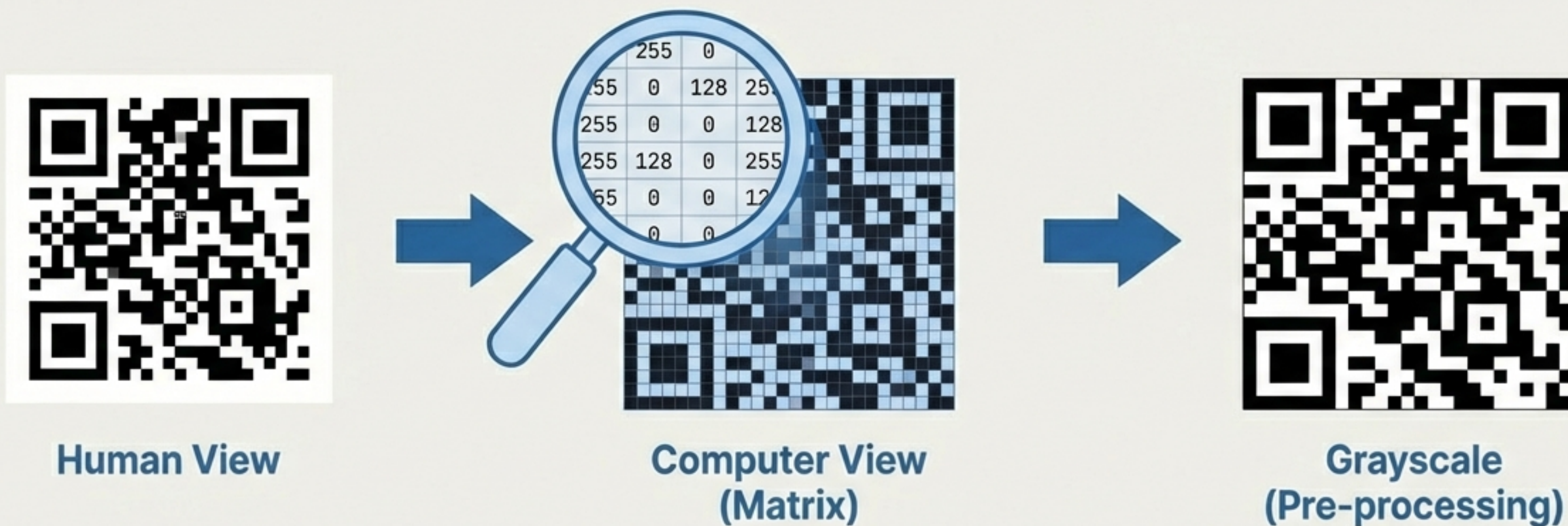
복잡한 자바스크립트 없이, 단 한 줄의 코드로 웹캠 스트림을 제어합니다.

```
1 import streamlit as st
2
3 img_file_buffer = st.camera_input('QR 코드를 비춰주세요')
```



💡 **Key Takeaway:** `st.camera\_input`은 단순히 화면만 보여주는 것이 아니라, 캡처된 순간의 이미지 데이터를 메모리 버퍼(Buffer) 형태로 반환합니다.

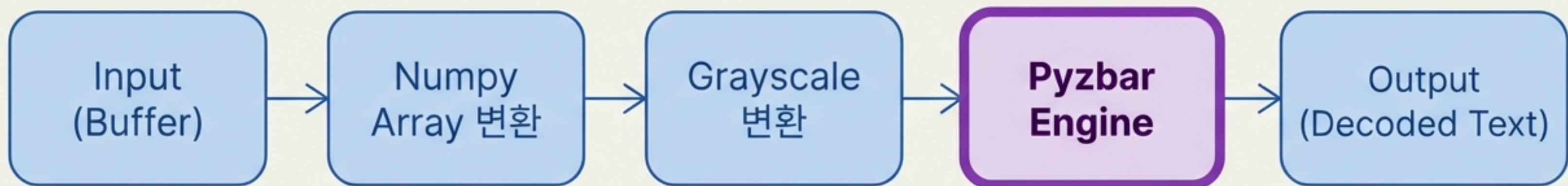
Insight: 기계가 세상을 보는 법



- 컴퓨터에게 이미지는 색상 값(0~255)을 가진 거대한 **행렬(Array)**일 뿐입니다.
- **전처리 (Pre-processing)**: 컬러 이미지는 정보량이 너무 많습니다. 우리는 흑백(Grayscale)으로 변환하여 연산 속도를 높이고 노이즈를 줄입니다.

💡 **Technical Note:** OpenCV의 ``cv2.imdecode``와 ``cv2.cvtColor``를 사용하여 원시 데이터를 '해석 가능한 숫자'로 바꿉니다.

Step 2: 해독 로직 구현



```
def process_image(image_file):  
    # ...  
    decoded_objects = pyzbar.decode(gray_img)  
    return decoded_objects
```

Result: <https://www.example.com>

Step 3: 사용자 경험(UX) 확장

모든 사용자가 웹캠을 사용할 수 있는 것은 아닙니다.

카메라로 스캔

이미지 파일 업로드



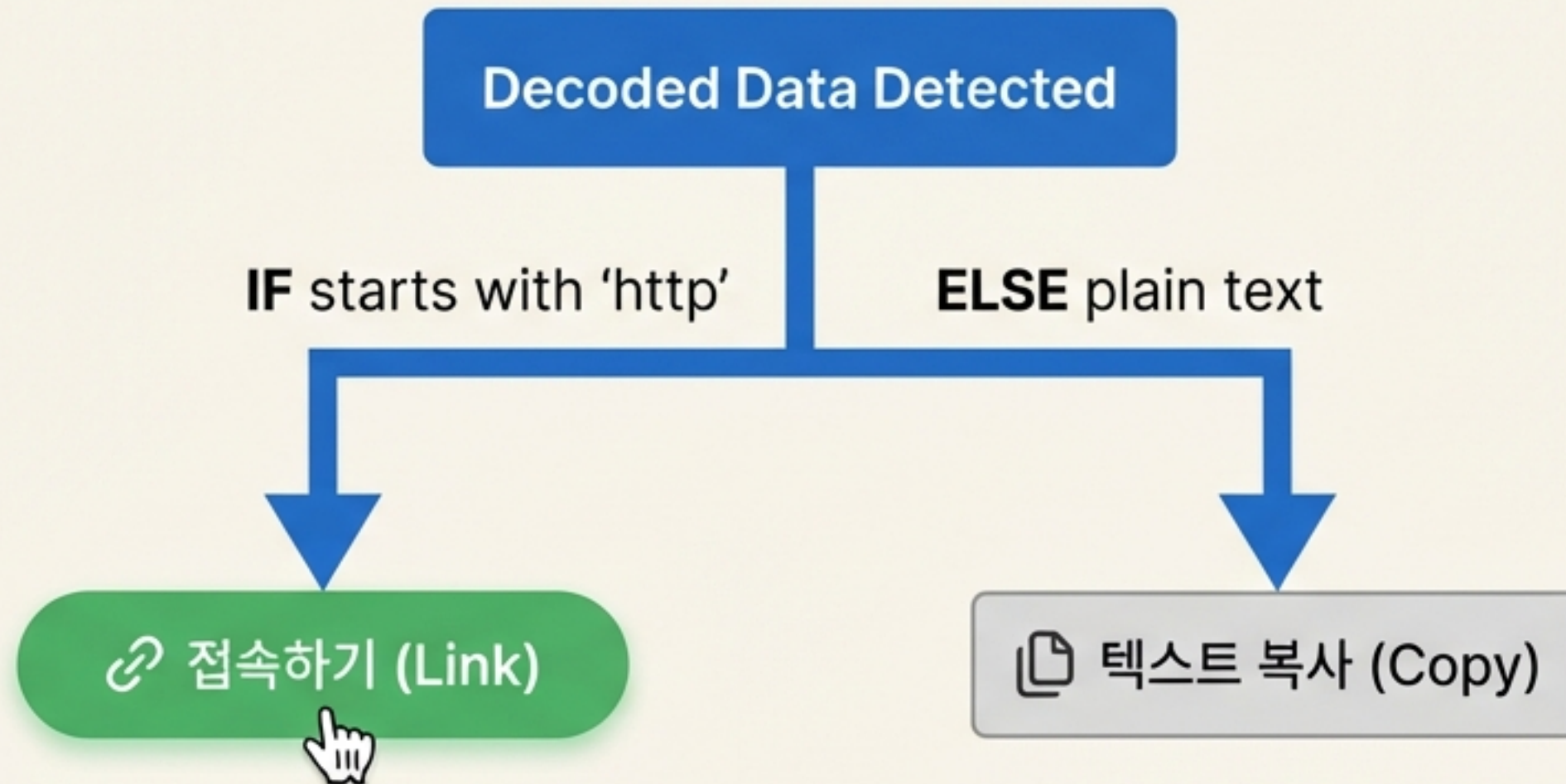
```
tab1, tab2 = st.tabs(['카메라로 스캔', '이미지 파일 업로드'])
```



Design Note: 탭 기능을 통해 UI 공간을 절약하고 사용자의 선택지를 명확하게 제시합니다.

Step 4: 데이터에 반응하는 스마트 연결

단순 텍스트 출력에서 멈추지 않고, 결과값의 **형태(Type)**에 따라 앱의 행동이 달라져야 합니다.



UX Insight: 데이터의 문맥을 파악하여 사용자가 다음에 취할 행동(Action)을 미리 제안합니다.

⚠️ 위기 발생: 현실 세계의 변수들



Ideal Condition



Real World (Fail)

조명이 어둡거나 각도가 기울어지면 기본 `pyzbar` 엔진만으로는 인식률이 급격히 떨어집니다.

The Mission: 코드를 수정하여 **로버스트(Robust)**한 인식 시스템을 만들어야 합니다.

해결책 1: 명암 대비 최적화 (CLAHE)

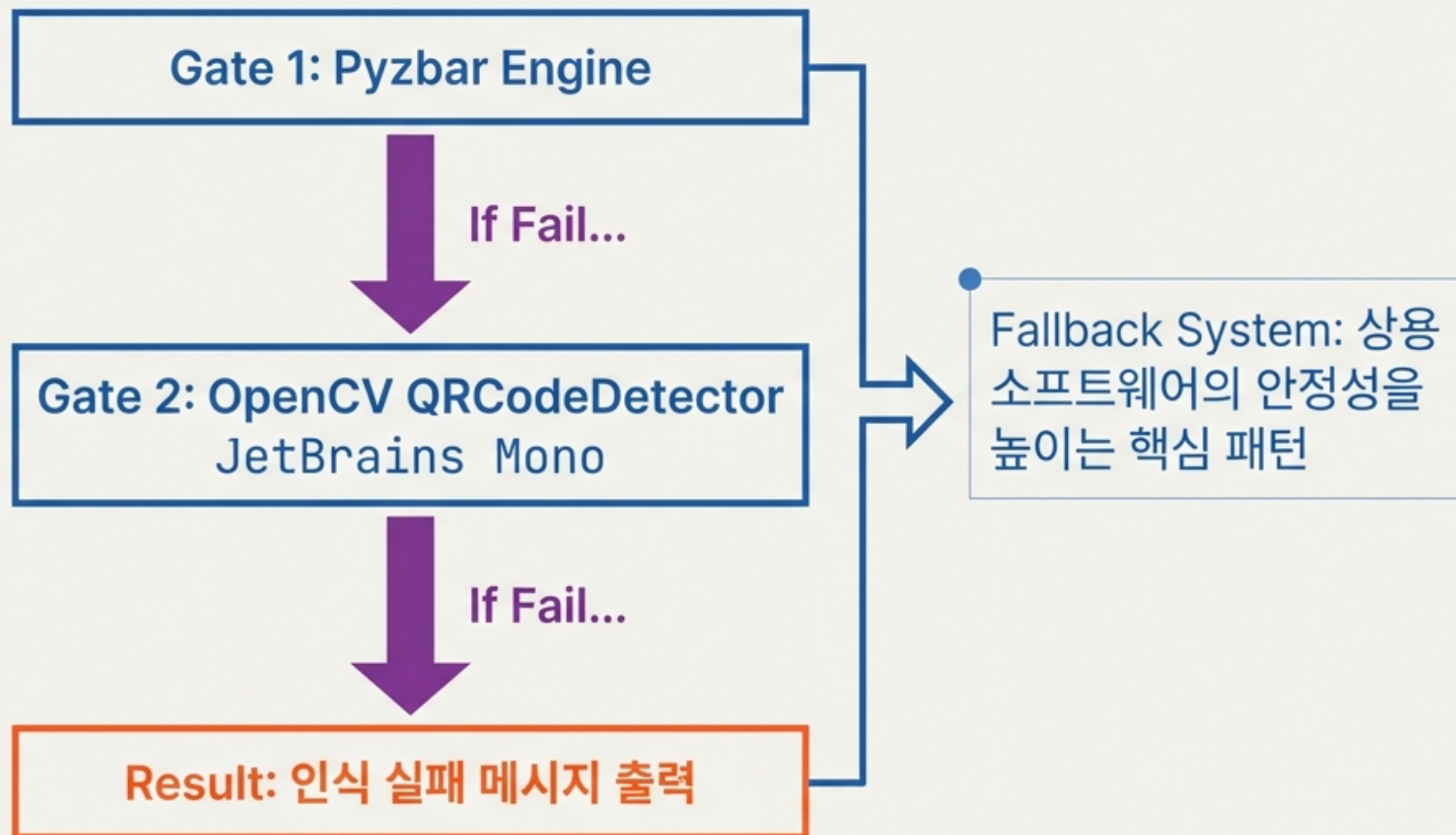


이미지의 어두운 영역과 밝은 영역의 대비를 극대화하여 QR 코드의 패턴을 선명하게 만듭니다.

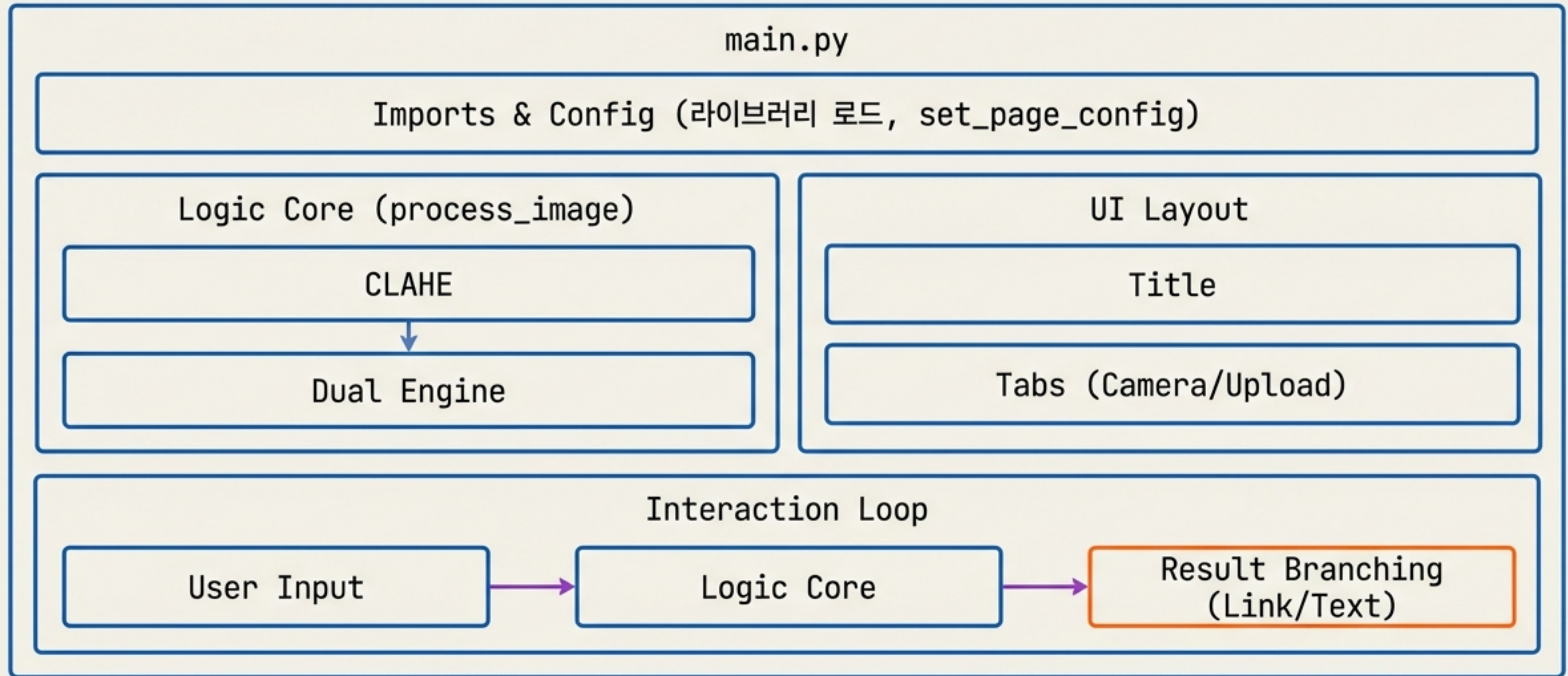
단순 히스토그램 균일화가 아닌, **CLAHE (Contrast Limited Adaptive Histogram Equalization)** 기법을 사용하여 노이즈 증폭을 막습니다.

해결책 2: 이중 엔진 시스템 (Dual Engine Fallback)

하나의 알고리즘에 의존하지 않습니다.



최종 아키텍처 리뷰



2~3시간 만에 읽기 전용 스크립트에서 시각적 인공지능 웹앱으로 진화했습니다.

Next Level: 무엇을 더 할 수 있을까요?

Your Journey

- ✓ 기초: 카메라 연동 및 데이터 스트림 이해
- ✓ UX: 탭 기능과 사용자 편의성 증대
- ✓ 심화: CLAHE와 Fallback 로직을 통한 엔지니어링 문제 해결

Challenges



스캔 기록을 데이터베이스(SQLite)에 저장해볼까요?



바코드 상품명을 API로 검색해볼까요?

**“완벽한 앱은 없습니다.
끊임없이 개선하는 엔지니어만 있을 뿐입니다.”**